Commands used to generate logs

Matteo Franzil, Valentino Armani, Luis Augusto Dias Knob

2024-05-28

Contents

1 Ignored logs

1.1 Ignored namespaces and users

The following namespaces are ignored:

- `falco`
- `kube-flannel`

The following users/SAs are ignored:

- `system:serviceaccount:kube-flannel:flannel`

1.2 Endpoints different than api/apis

We will ignore endpoints which are not the regular API ones.

- Verb: any
- Users: any
- Objects: any
- Endpoints: any other than `/api` or `/apis`

1.3 API Server watching objects

The API server monitors core components with a 10minute timeout. It remains unclear what the API server watches in other groups: we probably need to deploy more components to see what the API server watches.

- Verb: `watch`
- Users: `system:apiserver`
- Objects: `configmaps`, `clusterroles`, `namespaces`, `serviceaccounts`, `resourcequotas`, `clusterrolebindi` `rolebindings`, `secrets`, `nodes`, `pods`, `roles`
- Namespaces: none, apart from `configmaps` in `kube-system`. Additionally, if legacy service accounts areused, the API server also watches `kube-apiserver-legacy-service-account-token-tracking`, a CM in `kube-system`.

1.4 `kube-controller-manager` watching objects

The `kube-system` namespace watches objects, similar to the API server. However, ConfigMaps are watched over non-namespaced objects and also `certificateigningrequests` are watched.

I believe that the objects watched by the Kube Controller Manager are specific to what is deployed in the cluster. For example, if the cluster has a deployment, the Kube Controller Manager will watch also `deployments`.

- Verb: `watch`
- Users: `system:kube-controller-manager`
- Objects: same as the API server, plus `certificateigningrequests`
- Namespaces: none

1.5 `kube-controller-manager` getting and creating tokens for GC and RQ controllers

The `kube-controller-manager` gets every less than an hour the serviceaccounts `/api/v1/namespaces/kube-syst` and `/api/v1/namespaces/kube-system/serviceaccounts/resourcequota-controller`. It then creates tokens for them, which expire in an hour.

1.5.1 1

- Verb: `get`
- Users: `system:kube-controller-manager`, groups `system:authenticated`
- Objects: `/api/v1/namespaces/kube-system/serviceaccounts/generic-garbage-collector`

1.5.2 2

- Verb: `get`
- Users: `system:kube-controller-manager`, groups `system:authenticated`
- Objects: `/api/v1/namespaces/kube-system/serviceaccounts/resourcequota-controller`

1.5.3 3

- Verb: `create`
- Users: `system:kube-controller-manager`, groups `system:authenticated`
- Objects: `/api/v1/namespaces/kube-system/serviceaccounts/generic-garbage-collector/token`

1.5.4 4

- Verb: `create`
- Users: `system:kube-controller-manager`, groups `system:authenticated`
- Objects: `/api/v1/namespaces/kube-system/serviceaccounts/resourcequota-controller/token`

1.6 Kube Scheduler watching objects

The Kube Scheduler watches pods, nodes, namespaces, always with a 10-minute timeout.

- Verb: `watch`
- Users: `system:kube-scheduler`
- Objects: `pods`, `nodes`, `namespaces`
- Namespaces: none

1.6.1 Extension-apiserver-authentication

This object is also watched by the Kube Scheduler.

- Verb: `watch`
- Users: `system:kube-scheduler`
- Object: `configmaps/extension-apiserver-authentication`
- Namespaces: `kube-system`

1.7 Nodes watching objects

Each node in the cluster watches pods (non-namespaced), nodes (themselves), and some configmaps.

1.7.1 Pods

- Verb: `watch`
- Users: `system:node:.*`, groups `["system:nodes","system:authenticated"]`
- Objects: `pods`
- Namespaces: none

1.7.2 Nodes

- Verb: `watch`
- Users: `system:node:.*`, groups `["system:nodes","system:authenticated"]`
- Objects: `nodes/{node-name}`
- Namespaces: none

1.7.3 ConfigMaps

Every node hosting some Pod of any namespace will be in charge of watching the serviceaccounts of the Pods that are running.

The CMs being watched are `kube-flannel-cfg`, `kube-proxy`, `kube-root-ca.crt`, and `coredns`. They of course depend on the components deployed in the cluster. Let's keep this generic.

- Verb: `watch`
- Users: `system:node:.*`, groups `["system:nodes","system:authenticated"]`
- Objects: `configmaps/kube-system/configmaps/{serviceaccount}`
- Namespace: `kube-system`

1.8 Nodes creating tokens

1.8.1 Nodes creating tokens for SAs in the kube-system namespace

Since nodes watch the configmaps of the Pods they are running, as the tokens of their service accounts expire, they will recreate them.

- Verb: `create`
- Users: `system:node:.*`, groups `["system:nodes","system:authenticated"]`
- Objects: `/api/v1/namespaces/kube-system/serviceaccounts/{serviceaccount}/token`

1.9 Nodes getting their own status

Nodes poll their own status every ten seconds.

- Verb: `get`
- Users: `system:node:.*`, groups `["system:nodes","system:authenticated"]`
- Objects: `/api/v1/nodes/{the same node}`

1.10 Nodes patching their status to update conditions

Nodes update their status (e.g., MemoryPressure, DiskPressure, PIDPressure) by patching their own status every five minutes.

- Verb: `patch`
- Users: `system:node:.*`, groups `["system:nodes","system:authenticated"]`
- Objects: `/api/v1/nodes/{the same node}`

1.11 CoreDNS watching namespaces

- Verb: `watch`
- Users: `system:serviceaccount:kube-system:coredns`
- Objects: `namespaces`

1.12 Kube-proxy watching nodes

- Verb: `watch`
- Users: `system:serviceaccount:kube-system:kube-proxy`
- Objects: `nodes`

2 Log setups level 1

2.1 Namespace log `N1` - Simple namespace creation and deletion

- Create a namespace:

```
kubectl create namespace rising
```

- List the namespaces:

```
kubectl get namespaces
```

- Get the namespace using `-o yaml`:

```
kubectl get namespace rising -o yaml
```

- Describe the namespace:

```
kubectl describe namespace rising
```

- Delete the namespace:

```
kubectl delete namespace rising
```

2.2 Namespace log `N2` - Namespace with resource quotas

- **Prerequisite**: recreate the `rising` namespace:

```
kubectl create namespace rising
```

- Create a ResourceQuota for the namespace:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: ResourceQuota
    metadata:
      name: rising-quota
    spec:
      hard:
        pods: "1"
        requests.cpu: "1"
        requests.memory: 1Gi
        limits.cpu: "2"
        limits.memory: 2Gi
    EOF
```

- Inspect the ResourceQuota:

```
kubectl -n rising describe resourcequota rising-quota
```

- Create a Pod that will exceed the quota:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Pod
    metadata:
      name: over-quota-pod
      namespace: rising
    spec:
      containers:
      - name: over-quota-container
        image: nginx
        resources:
          requests:
            cpu: "1"
            memory: 1.5Gi
          limits:
            cpu: "2"
            memory: 2Gi
        command: ["sh", "-c", "echo Hello, Kubernetes! && sleep 3600"]
    EOF
```

- Now create a Pod that will respect the quota:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Pod
    metadata:
      name: within-quota-pod
      namespace: rising
    spec:
      containers:
      - name: within-quota-container
        image: nginx
        resources:
          requests:
            cpu: "1"
            memory: 1Gi
          limits:
            cpu: "2"
            memory: 2Gi
        command: ["sh", "-c", "echo Hello, Kubernetes! && sleep 3600"]
    EOF
```

- Run a further Pod, it will fail:

```
kubectl -n rising run --image=nginx nginx
```

- Clean everything up, one pod will fail because it was never created:

```
kubectl -n rising delete pod over-quota-pod
kubectl -n rising delete pod within-quota-pod
kubectl delete resourcequota rising-quota
```

2.3 Pod log `P1` - Simple Pod creation and inspection

- Create a pod:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Pod
    metadata:
      name: rising-pod
      namespace: rising
    spec:
      containers:
      - name: first-container
        image: nginx
      - name: second-container
        image: alpine
        command: ["sh", "-c", "echo Hello, Kubernetes! && sleep 3600"]
    EOF
```

- Inspect the Pod:

```
kubectl -n rising describe pod rising-pod
```

- Patch the Pod and try to add a third container, this will fail:

```
kubectl -n rising patch pod rising-pod --type='json' -p='[{"op": "add", "path":
↪  "/spec/containers/1", "value": {"name": "third-container", "image": "nginx"}}]'
```

- Get the pod log:

```
kubectl -n rising logs rising-pod -c second-container
```

- Execute commands in the second containers, the first will complain about the shell, the second will work:

```
kubectl -n rising exec -t rising-pod -c second-container -- /bin/bash -c "echo Hello again,
↪  Kubernetes!"
kubectl -n rising exec -t rising-pod -c second-container -- /bin/sh -c "echo Hello again,
↪  Kubernetes!"
```

- Delete the pod:

```
kubectl -n rising delete pod rising-pod
```

2.4 Pod log `P2` - Image download

- **Prerequisite**: run the following helper script to wipe the image cache on all nodes. You **must** create the file `scripts/cluster-setup.sshconfig` with the SSH configuration for the nodes. Refer to the example in the same directory to get started.

```
for node in $(kubectl get nodes -o jsonpath='{.items[*].metadata.name}'); do
  echo "Pruning images on $node"
  ssh -F scripts/cluster-setup.sshconfig $node "sudo crictl rmi --prune"
done
```

- Create a Pod with a container that uses a non-existent image:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Pod
    metadata:
      name: image-pod
      namespace: rising
    spec:
      containers:
      - name: image-container
        image: www.fbk.eu/non-existent-image
        command: ["sh", "-c", "echo Hello, Kubernetes! && sleep 3600"]
    EOF
```

- Get the pod log:

```
kubectl -n rising logs image-pod -c image-container
```

- Get the events that led to the failure:

```
kubectl get events -n rising --sort-by='.metadata.creationTimestamp' --selector
↪  'involvedObject.name=image-pod'
```

- Delete the pod:

```
kubectl -n rising delete pod image-pod
```

- Create a pod with an image that will surely not be in cache:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Pod
    metadata:
      name: image-pod
      namespace: rising
    spec:
      containers:
      - name: image-container
        image: busybox
        command: ["sh", "-c", "echo Hello, Kubernetes! && sleep 3600"]
    EOF
```

- Wait for the Pod to be downloaded and monitor the events:

```
kubectl wait --for=condition=Ready pod/image-pod -n rising
kubectl get events -n rising --sort-by='.metadata.creationTimestamp'
```

- Delete the pod:

```
kubectl -n rising delete pod image-pod
```

2.5 ConfigMap `C1` - Create a ConfigMap and use it in a Pod, then delete it

- Create a ConfigMap:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: ConfigMap
    metadata:
      name: my-config
      namespace: rising
    data:
      status: "decent"
    EOF
```

- Get the configmap

```
kubectl -n rising get configmap my-config -o yaml
```

- Deploy a Pod that uses the ConfigMap:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Pod
    metadata:
      name: config-pod
      namespace: rising
    spec:
      containers:
      - name: config-container
        image: alpine
        command: ["sh", "-c", 'echo Status is \$STATUS && sleep 3600']
        env:
        - name: STATUS
          valueFrom:
            configMapKeyRef:
              name: my-config
              key: status
    EOF
```

- Access the logs of the Pod:

```
kubectl wait --for=condition=Ready pod/config-pod -n rising
kubectl -n rising logs config-pod -c config-container
```

- Clean everything up:

```
kubectl -n rising delete pod config-pod
kubectl -n rising delete configmap my-config
```

2.6 Secret `S1` - Create a Secret and use it as environment in a Pod, then delete it

- Create a secret

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Secret
    metadata:
      name: my-secret
      namespace: rising
    type: Opaque
    data:
      username: $(echo -n "admin" | base64)
      password: $(echo -n "password" | base64)
    EOF
```

- Inspect the secret:

```
kubectl -n rising get secret my-secret -o yaml
```

- Attach the secret to a Pod:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Pod
    metadata:
      name: secret-pod
      namespace: rising
    spec:
      containers:
      - name: secret-container
        image: alpine
        command: ["sh", "-c", 'echo Username is \$USERNAME && echo Password is \$PASSWORD &&
    sleep 3600']
        env:
        - name: USERNAME
          valueFrom:
            secretKeyRef:
              name: my-secret
              key: username
        - name: PASSWORD
          valueFrom:
            secretKeyRef:
              name: my-secret
              key: password
    EOF
```

- Inspect the Pod:

```
kubectl -n rising describe pod secret-pod
```

- Access the logs of the Pod:

```
kubectl wait --for=condition=Ready pod/secret-pod -n rising
kubectl -n rising logs secret-pod -c secret-container
```

- Clean everything up:

```
kubectl -n rising delete pod secret-pod
kubectl -n rising delete secret my-secret
```

2.7 Secret S2 - Create a Secret and mount it in a Pod

- Create a secret

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Secret
    metadata:
      name: my-secret-2
      namespace: rising
    type: Opaque
    data:
      username: $(echo -n "admin" | base64)
      password: $(echo -n "password" | base64)
    EOF
```

- Create a Pod, but now, mount the secret as a volume:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Pod
    metadata:
      name: secret-pod-2
      namespace: rising
    spec:
      containers:
      - name: secret-container-2
        image: alpine
        command: ["sh", "-c", 'echo Username is \$USERNAME && echo Password is \$PASSWORD &&
    sleep 3600']
        volumeMounts:
        - name: secret-volume
          mountPath: /etc/secret
      volumes:
      - name: secret-volume
        secret:
          secretName: my-secret-2
    EOF
```

- Inspect the Pod:

```
kubectl -n rising describe pod secret-pod-2
```

- Access the Secret from the Pod:

```
kubectl wait --for=condition=Ready pod/secret-pod-2 -n rising
kubectl -n rising exec secret-pod-2 -- cat /etc/secret/username
echo
kubectl -n rising exec secret-pod-2 -- cat /etc/secret/password
```

- Clean everything up:

```
kubectl -n rising delete pod secret-pod-2
kubectl -n rising delete secret my-secret-2
```

2.8 Node log `NO1` - Node inspection

- Assess the available nodes in the cluster:

```
kubectl get nodes
```

- Inspect one of the nodes:

```
kubectl describe node kubeadm-worker1
```

2.9 Certificate log `CSR1` - Create a CertificateSigningRequest and approve it

- Note: an helper script is available in `scripts/generate_k8s_user.sh` that automates what is done in this section.

- Use openssl to generate a key and a certificate signing request:

```
export TEMPDIR=$(mktemp -d)
export ORIGINDIR=$(pwd)
cd $TEMPDIR
export KUBEUSER=malicious-user
openssl genpkey -out ${KUBEUSER}.key -algorithm Ed25519
```

```
openssl req -new -key ${KUBEUSER}.key -out ${KUBEUSER}.csr -subj
↪  "/CN=${KUBEUSER}/O=rising/O=fbk/"
echo "Key created, available in ${TEMPDIR} for user ${KUBEUSER}"
```

- Create a CertificateSigningRequest:

```
cat <<EOF | kubectl apply -f -
    apiVersion: certificates.k8s.io/v1
    kind: CertificateSigningRequest
    metadata:
      name: ${KUBEUSER}
    spec:
      request: $(cat ${KUBEUSER}.csr | base64 | tr -d '\n')
      signerName: kubernetes.io/kube-apiserver-client
      usages:
      - client auth
    EOF
```

- Deny the CertificateSigningRequest:

```
kubectl certificate deny ${KUBEUSER}
```

- Delete the failed CSR and generate a new keypair for a legitimate user:

```
rm ${KUBEUSER}.csr ${KUBEUSER}.key
kubectl delete csr ${KUBEUSER}
export KUBEUSER=rising-user
openssl genpkey -out ${KUBEUSER}.key -algorithm Ed25519
openssl req -new -key ${KUBEUSER}.key -out ${KUBEUSER}.csr -subj
↪  "/CN=${KUBEUSER}/O=rising/O=fbk/"
echo "Key created, available in ${TEMPDIR} for user ${KUBEUSER}"
```

- Generate a new CertificateSigningRequest:

```
cat <<EOF | kubectl apply -f -
    apiVersion: certificates.k8s.io/v1
    kind: CertificateSigningRequest
    metadata:
      name: ${KUBEUSER}
    spec:
      request: $(cat ${KUBEUSER}.csr | base64 | tr -d '\n')
      signerName: kubernetes.io/kube-apiserver-client
      usages:
      - client auth
    EOF
```

- Inspect the status of the CSR:

```
kubectl get csr ${KUBEUSER} -o yaml
```

- Approve the CertificateSigningRequest:

```
kubectl certificate approve ${KUBEUSER}
```

- Obtain the certificate from the CSR and store it in the folder

```
kubectl get csr ${KUBEUSER} -o jsonpath='{.status.certificate}' | base64 -d > ${KUBEUSER}.crt
head -n 2 ${KUBEUSER}.crt
```

- Use the certificate to generate a `.kubeconfig` for the user:

```
cat <<EOF > ${KUBEUSER}.kubeconfig
    apiVersion: v1
    kind: Config
    preferences: {}
    clusters:
    $(kubectl config view --raw --minify --flatten | yq .clusters)
    contexts: []
    users: []
    EOF
kubectl --kubeconfig ${KUBEUSER}.kubeconfig config set-credentials ${KUBEUSER}
↪  --client-certificate=${KUBEUSER}.crt --client-key=${KUBEUSER}.key --embed-certs=true
kubectl --kubeconfig ${KUBEUSER}.kubeconfig config set-context ${KUBEUSER}
↪  --cluster=kubernetes --user=${KUBEUSER} --cluster=risinglab
kubectl --kubeconfig ${KUBEUSER}.kubeconfig config use-context ${KUBEUSER}
cp ${KUBEUSER}.kubeconfig ${ORIGINDIR}
```

2.10 Role log `R1` - Create a Role and bind it to a User

- **Prerequisite**: complete `CSR1` and obtain the credentials for the user. Make sure they are available as `rising-user.kubeconfig` in the current directory.
- Create a Role:

```
cat <<EOF | kubectl apply -f -
    apiVersion: rbac.authorization.k8s.io/v1
    kind: Role
    metadata:
      name: rising-reader
      namespace: rising
    rules:
    - apiGroups: [""]
      resources: ["pods"]
      verbs: ["get", "list"]
    EOF
```

- List the roles in the namespace:

```
kubectl -n rising get roles
```

- We changed idea: let's also give `get`, `list`, `patch` and `watch` for Secrets:

```
kubectl patch role rising-reader -n rising --type='json' -p='[{"op": "add", "path":
↪  "/rules/0", "value": {"apiGroups": [""], "resources": ["secrets"], "verbs": ["get",
↪  "list", "patch", "watch"]}}]'
```

- Let's create a RoleBinding and assign it to the user:

```
cat <<EOF | kubectl apply -f -
    apiVersion: rbac.authorization.k8s.io/v1
    kind: RoleBinding
    metadata:
      name: pod-reader-binding
      namespace: rising
    subjects:
    - kind: User
      name: rising-user
    roleRef:
      kind: Role
      name: rising-reader
      apiGroup: rbac.authorization.k8s.io
    EOF
```

- Test the permissions of the user:

```
kubectl --kubeconfig rising-user.kubeconfig get pods -n rising
kubectl --kubeconfig rising-user.kubeconfig get secrets -n rising
kubectl --kubeconfig rising-user.kubeconfig delete pod non-existent-pod -n rising
```

2.11 Role log `R2` - Create a ClusterRole and bind it to a Group

- **Prequisite**: complete `CSR1` and obtain the credentials for the user. Make sure they are available as `rising-user.kubeconfig` in the current directory.
- **Prequisite**: complete `R1` and keep the `rising-reader` Role.
- Create a ClusterRole:

```
cat <<EOF | kubectl apply -f -
    apiVersion: rbac.authorization.k8s.io/v1
    kind: ClusterRole
    metadata:
      name: cluster-reader
    rules:
    - apiGroups: [""]
      resources: ["pods"]
      verbs: ["get", "list"]
    - apiGroups: [""]
      resources: ["secrets"]
      verbs: ["get", "list", "patch", "watch"]
    EOF
```

- Actually, let's straight out replace the ClusterRole with a new one:

```
cat <<EOF | kubectl apply -f -
    apiVersion: rbac.authorization.k8s.io/v1
    kind: ClusterRole
    metadata:
      name: cluster-reader
    rules:
    - apiGroups: [""]
      resources: ["nodes"]
      verbs: ["get", "list"]
    EOF
```

- Create a ClusterRoleBinding and assign it to the `fbk` group:

```
cat <<EOF | kubectl apply -f -
    apiVersion: rbac.authorization.k8s.io/v1
    kind: ClusterRoleBinding
    metadata:
      name: cluster-reader-binding
    subjects:
      - kind: Group
        name: fbk
    roleRef:
      kind: ClusterRole
      name: cluster-reader
      apiGroup: rbac.authorization.k8s.io
    EOF
```

- Test the permissions of the user:

```
kubectl --kubeconfig rising-user.kubeconfig get nodes
kubectl --kubeconfig rising-user.kubeconfig get pods -n rising
kubectl --kubeconfig rising-user.kubeconfig get configmaps -n rising
kubectl --kubeconfig rising-user.kubeconfig get pods -n kube-system
```

- Clean everything up:

```
kubectl delete rolebinding pod-reader-binding -n rising
kubectl delete role rising-reader -n rising
kubectl delete clusterrolebinding cluster-reader-binding
kubectl delete clusterrole cluster-reader
kubectl delete csr rising-user
```

2.12 Role log R3 - Create a ServiceAccount for a long-running Pod and bind it to a Role

- Set up a ServiceAccount:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: ServiceAccount
    metadata:
      name: long-running-sa
      namespace: rising
    EOF
```

- Create a Role:

```
cat <<EOF | kubectl apply -f -
    apiVersion: rbac.authorization.k8s.io/v1
    kind: Role
    metadata:
      name: long-running-role
      namespace: rising
    rules:
    - apiGroups: [""]
      resources: ["pods"]
      verbs: ["get", "list"]
    EOF
```

- Bind the ServiceAccount to the Role:

```
cat <<EOF | kubectl apply -f -
    apiVersion: rbac.authorization.k8s.io/v1
    kind: RoleBinding
    metadata:
      name: long-running-role-binding
      namespace: rising
    subjects:
    - kind: ServiceAccount
      name: long-running-sa
      namespace: rising
    roleRef:
      kind: Role
      name: long-running-role
      apiGroup: rbac.authorization.k8s.io
    EOF
```

- Create a Pod that uses the ServiceAccount:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Pod
    metadata:
      name: long-running-pod
      namespace: rising
    spec:
      serviceAccountName: long-running-sa
      containers:
      - name: long-running-container
        image: ubuntu
        command: ["sh", "-c", "echo Hello, Kubernetes! && sleep 3600"]
    EOF
```

- Inspect the Pod:

```
kubectl -n rising describe pod long-running-pod | head -n 10
```

- Try to use the ServiceAccount from inside the Pod:

```
kubectl wait --for=condition=Ready pod/long-running-pod -n rising
kubectl -n rising exec long-running-pod -- sh -c 'apt-get -qq update; apt-get install -yqq
    curl yq >/dev/null;
    export KUBE_TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token);
    curl -sSk -H "Authorization: Bearer $KUBE_TOKEN"
    "https://$KUBERNETES_SERVICE_HOST:$KUBERNETES_PORT_443_TCP_PORT/apis/authentication.k8s.io/v1/selfsubjec
    > temp.json; jq .message temp.json
    curl -sSk -H "Authorization: Bearer $KUBE_TOKEN"
↪   "https://$KUBERNETES_SERVICE_HOST:$KUBERNETES_PORT_443_TCP_PORT/api/v1/namespaces/rising/pods"
↪   > temp.json; head -n 5 temp.json'
```

- Clean everything up:

```
kubectl -n rising delete pod long-running-pod
kubectl -n rising delete role long-running-role
kubectl -n rising delete rolebinding long-running-role-binding
kubectl -n rising delete serviceaccount long-running-sa
```

2.13 Access Review AR1 - Use can-i to view permissions of self, other users, over different namespaces

- Create a set of ServiceAccounts for the logs (while this should not be the case in a real-world scenario, it is done since SAs and Users are equivalent once authorized):

```
kubectl apply -f - <<EOF
    apiVersion: v1
    kind: ServiceAccount
    metadata:
      name: alice
      namespace: rising
    ---
    apiVersion: v1
    kind: ServiceAccount
    metadata:
      name: bob
      namespace: rising
    ---
    apiVersion: v1
    kind: ServiceAccount
    metadata:
      name: charlie
      namespace: rising
    EOF
```

- Create tokens for the ServiceAccounts:

```
kubectl apply -f - <<EOF
    apiVersion: v1
    kind: Secret
    metadata:
      name: alice-token
      namespace: rising
      annotations:
        kubernetes.io/service-account.name: alice
    type: kubernetes.io/service-account-token
    ---
    apiVersion: v1
    kind: Secret
    metadata:
      name: bob-token
      namespace: rising
      annotations:
        kubernetes.io/service-account.name: bob
    type: kubernetes.io/service-account-token
    ---
    apiVersion: v1
    kind: Secret
    metadata:
      name: charlie-token
      namespace: rising
      annotations:
        kubernetes.io/service-account.name: charlie
    type: kubernetes.io/service-account-token
    ---
    EOF
```

- Create two Roles, one for reading Pods and one for reading Secrets:

```
kubectl apply -f - <<EOF
    apiVersion: rbac.authorization.k8s.io/v1
    kind: Role
    metadata:
      name: pod-reader
      namespace: rising
    rules:
    - apiGroups: [""]
      resources: ["pods"]
      verbs: ["get", "list"]
    ---
    apiVersion: rbac.authorization.k8s.io/v1
    kind: Role
    metadata:
      name: secret-reader
      namespace: rising
    rules:
    - apiGroups: [""]
      resources: ["secrets"]
      verbs: ["get", "list"]
    EOF
```

- Bind the ServiceAccounts to the Roles:

```
kubectl apply -f - <<EOF
    apiVersion: rbac.authorization.k8s.io/v1
    kind: RoleBinding
    metadata:
      name: pod-reader-binding
      namespace: rising
    subjects:
      - kind: ServiceAccount
        name: alice
        namespace: rising
      - kind: ServiceAccount
        name: bob
        namespace: rising
    roleRef:
      kind: Role
      name: pod-reader
      apiGroup: rbac.authorization.k8s.io
    ---
    apiVersion: rbac.authorization.k8s.io/v1
    kind: RoleBinding
    metadata:
      name: secret-reader-binding
      namespace: rising
    subjects:
      - kind: ServiceAccount
        name: charlie
        namespace: rising
    roleRef:
      kind: Role
      name: secret-reader
      apiGroup: rbac.authorization.k8s.io
    EOF
```

- Obtain credentials for the ServiceAccounts and generate a kubeconfig for them:

```
kubectl get secret alice-token -n rising -o jsonpath='{.data.token}' | base64 -d > alice.token
kubectl get secret bob-token -n rising -o jsonpath='{.data.token}' | base64 -d > bob.token
kubectl get secret charlie-token -n rising -o jsonpath='{.data.token}' | base64 -d >
↪  charlie.token
```

```
cat <<EOF > alice.kubeconfig
    apiVersion: v1
    kind: Config
    preferences: {}
    clusters:
    $(kubectl config view --raw --minify --flatten | yq .clusters)
    contexts: []
    users: []
    EOF
cp alice.kubeconfig bob.kubeconfig
cp alice.kubeconfig charlie.kubeconfig
kubectl --kubeconfig alice.kubeconfig config set-credentials alice --token=$(cat alice.token)
kubectl --kubeconfig bob.kubeconfig config set-credentials bob --token=$(cat bob.token)
kubectl --kubeconfig charlie.kubeconfig config set-credentials charlie --token=$(cat
↪  charlie.token)
kubectl --kubeconfig alice.kubeconfig config set-context alice --cluster=kubernetes
↪  --user=alice --cluster=risinglab
kubectl --kubeconfig bob.kubeconfig config set-context bob --cluster=kubernetes --user=bob
↪  --cluster=risinglab
kubectl --kubeconfig charlie.kubeconfig config set-context charlie --cluster=kubernetes
↪  --user=charlie --cluster=risinglab
kubectl --kubeconfig alice.kubeconfig config use-context alice
kubectl --kubeconfig bob.kubeconfig config use-context bob
kubectl --kubeconfig charlie.kubeconfig config use-context charlie
rm alice.token bob.token charlie.token
```

- Check as Alice if she can list Pods and Secrets in the namespace:

```
kubectl --kubeconfig alice.kubeconfig auth can-i list pods -n rising
kubectl --kubeconfig alice.kubeconfig auth can-i list secrets -n rising
```

- Check as Bob if he can list Pods in the namespaces `rising` and `kube-system`:

```
kubectl --kubeconfig bob.kubeconfig auth can-i list pods -n rising
kubectl --kubeconfig bob.kubeconfig auth can-i list pods -n kube-system
```

- Check as Charlie if he can list Secrets in the namespace:

```
kubectl --kubeconfig charlie.kubeconfig auth can-i list secrets -n rising
```

- Check as Charlie if he can list nodes:

```
kubectl --kubeconfig charlie.kubeconfig auth can-i list nodes
```

- Check what Charlie can do in the `rising` namespace:

```
kubectl --kubeconfig charlie.kubeconfig auth can-i --list -n rising
```

2.14 Access Review `AR2` - Use auth whoami to view the current user

- **Prerequisite**: complete `AR1` and obtain the credentials for the users. Make sure they are available as `alice.kubeconfig`, `bob.kubeconfig` and `charlie.kubeconfig` in the current directory.

- Check as each of the users who they are:

```
kubectl --kubeconfig alice.kubeconfig auth whoami
kubectl --kubeconfig bob.kubeconfig auth whoami
kubectl --kubeconfig charlie.kubeconfig auth whoami
```

2.15 Access Review `AR3` - Impersonating a user

- **Prerequisite**: complete `AR1` and obtain the credentials for the users. Make sure they are available as `alice.kubeconfig`, `bob.kubeconfig` and `charlie.kubeconfig` in the current directory.
- Augment Charlie's capabilities to let him impersonate ServiceAccounts:

```
kubectl apply -f - <<EOF
    apiVersion: rbac.authorization.k8s.io/v1
    kind: ClusterRole
    metadata:
      name: impersonator
    rules:
      - apiGroups: [""]
        resources: ["serviceaccounts"]
        verbs: ["impersonate"]
        resourceNames: ["alice", "bob"]
    ---
    apiVersion: rbac.authorization.k8s.io/v1
    kind: ClusterRoleBinding
    metadata:
      name: impersonator-binding
    subjects:
      - kind: ServiceAccount
        name: charlie
        namespace: rising
    roleRef:
      kind: ClusterRole
      name: impersonator
      apiGroup: rbac.authorization.k8s.io
    EOF
```

- Check if Charlie can impersonate any user or SA:

```
kubectl --kubeconfig charlie.kubeconfig auth can-i impersonate users
kubectl --kubeconfig charlie.kubeconfig auth can-i impersonate serviceaccounts
```

- Let Charlie see if Alice can list Pods in `rising`, and what she can do in `kube-system`:

```
kubectl --kubeconfig charlie.kubeconfig auth can-i list pods -n rising --as
↪  system:serviceaccount:rising:alice
kubectl --kubeconfig charlie.kubeconfig auth can-i --list -n kube-system --as
↪  system:serviceaccount:rising:alice
```

- As Charlie, let's impersonate Bob and create a Pod in the `rising` namespace, then list Pods:

```
kubectl --kubeconfig charlie.kubeconfig --as system:serviceaccount:rising:bob run
↪  --image=ubuntu --restart=Never --namespace=rising --command -- sleep 3600
kubectl --kubeconfig charlie.kubeconfig --as system:serviceaccount:rising:bob get pods -n
↪  rising
```

- As Charlie, let's impersonate Alice and try to list nodes:

```
kubectl --kubeconfig charlie.kubeconfig --as system:serviceaccount:rising:alice get nodes
```

- Clean everything up:

```
kubectl delete clusterrolebinding impersonator-binding
kubectl delete clusterrole impersonator
kubectl delete rolebinding pod-reader-binding -n rising
kubectl delete role pod-reader -n rising
kubectl delete rolebinding secret-reader-binding -n rising
```

```
kubectl delete role secret-reader -n rising
kubectl delete secret alice-token bob-token charlie-token -n rising
kubectl delete serviceaccount alice bob charlie -n rising
rm alice.kubeconfig bob.kubeconfig charlie.kubeconfig
```

3 Log setups level 2

3.1 Deployment log D1 - Create a Deployment for a ReplicaSet and expose it as a Service

- Create a Deployment:

```
cat <<EOF | kubectl apply -f -
    apiVersion: apps/v1
    kind: Deployment
    metadata:
      name: test-deployment
      namespace: rising
      labels:
        app: test
    spec:
      replicas: 2
      selector:
        matchLabels:
          app: test
      template:
        metadata:
          labels:
            app: test
       spec:
          containers:
          - name: nginx
            image: nginx:latest
            ports:
            - containerPort: 80
    EOF
```

- Inspect the Deployment:

```
kubectl get deployments test-deployment -n rising
```

- Inspect the rollout status:

```
kubectl rollout status deployment test-deployment -n rising
```

- Inspect the ReplicaSet created by the Deployment:

```
kubectl get replicasets -l app=test -n rising
```

- Describe the ReplicaSet directly:

```
kubectl describe replicasets -l app=test -n rising
```

- Expose the Deployment as a Service:

```
kubectl expose deployment test-deployment --type=NodePort --port=80 --target-port=80
↪  --name=test-deployment -n rising
```

- Inspect the Service:

```
kubectl get services test-deployment -n rising
```

- Tear everything down:

```
kubectl delete service test-deployment -n rising
kubectl delete deployment test-deployment -n rising
```

3.2 Deployment log D2 - Scale a Deployment and update its image

- Create a Deployment again:

```
cat <<EOF | kubectl apply -f -
    apiVersion: apps/v1
    kind: Deployment
    metadata:
      name: test-deployment
      namespace: rising
      labels:
        app: test
    spec:
      replicas: 2
      selector:
        matchLabels:
          app: test
      template:
        metadata:
          labels:
            app: test
       spec:
          containers:
          - name: nginx
            image: nginx:latest
            ports:
            - containerPort: 80
    EOF
```

- Scale the Deployment to 3 replicas:

```
kubectl scale deployment test-deployment --replicas=3 -n rising
```

- Inspect the Deployment:

```
kubectl get deployments test-deployment -n rising
```

- Update the image of the Deployment:

```
kubectl set image deployment test-deployment nginx=nginx:1.16.1 -n rising
```

- Inspect the rollout status:

```
kubectl rollout status deployment test-deployment -n rising
```

- Scale down the Deployment to 2 replicas:

```
kubectl scale deployment test-deployment --replicas=2 -n rising
```

- Inspect ReplicaSets:

```
kubectl get replicasets -l app=test -n rising
```

- Inspect the Pods:

```
kubectl get pods -l app=test -n rising
```

- Tear everything down:

```
kubectl delete deployment test-deployment -n rising
```

3.3 Deployment log D3 - Roll back, pause and resume a Deployment

- Create a Deployment again:

```
cat <<EOF | kubectl apply -f -
    apiVersion: apps/v1
    kind: Deployment
    metadata:
      name: test-deployment
      namespace: rising
      labels:
        app: test
    spec:
      replicas: 2
      selector:
        matchLabels:
          app: test
      template:
        metadata:
          labels:
            app: test
       spec:
          containers:
          - name: nginx
            image: nginx:latest
            ports:
            - containerPort: 80
    EOF
```

- Wait for the Deployment to be ready:

```
kubectl wait --for=condition=Available deployment/test-deployment -n rising
```

- Pause the Deployment:

```
kubectl rollout pause deployment test-deployment -n rising
```

- Update the image of the Deployment:

```
kubectl set image deployment test-deployment nginx=nginx:1.16.1 -n rising
```

- Inspect the rollout history:

```
kubectl rollout history deployment test-deployment -n rising
```

- Inspect the ReplicaSets:

```
kubectl get replicasets -l app=test -n rising
```

- Resume the Deployment:

```
kubectl rollout resume deployment test-deployment -n rising
```

- Inspect the rollout history again:

```
kubectl rollout history deployment test-deployment -n rising
```

- Roll back the Deployment to the previous revision:

```
kubectl rollout undo deployment test-deployment -n rising
```

- Inspect the rollout history one more time:

```
kubectl rollout history deployment test-deployment -n rising
```

- List Pods:

```
kubectl get pods -l app=test
```

- Tear everything down:

```
kubectl delete deployment test-deployment -n rising
```

3.4 Deployment log D4 - Watch a Deployment fail and recover

- Install a restrictive ResourceQuota:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: ResourceQuota
    metadata:
      name: test-quota
      namespace: rising
    spec:
      hard:
        pods: "1"
    EOF
```

- Create a Deployment again:

```
cat <<EOF | kubectl apply -f -
    apiVersion: apps/v1
    kind: Deployment
    metadata:
      name: test-deployment
      namespace: rising
      labels:
        app: test
    spec:
      replicas: 2
      selector:
        matchLabels:
          app: test
      template:
        metadata:
          labels:
            app: test
        spec:
          containers:
          - name: nginx
            image: nginx:latest
            ports:
            - containerPort: 80
    EOF
```

- Check the status of the Deployment:

```
kubectl describe deployment test-deployment -n rising
```

- Relax the ResourceQuota:

```
kubectl patch resourcequota test-quota -p '{"spec":{"hard":{"pods":"5"}}}' -n rising
```

- Check the status of the Deployment again:

```
kubectl rollout status deployment test-deployment -n rising
```

- Change the image of the Deployment and put a non-existent image:

```
kubectl set image deployment test-deployment nginx=engincs:ahahahah -n rising
```

- Check the status of the Deployment:

```
kubectl describe deployment test-deployment -n rising
```

- List the Pods:

```
kubectl get pods -l app=test -n rising
```

- Fix the image of the Deployment:

```
kubectl set image deployment test-deployment nginx=nginx:1.16.1 -n rising
```

- Check the status again:

```
kubectl get deployment test-deployment -n rising
```

- Now, set an impossible affinity for the Deployment:

```
kubectl patch deployment test-deployment -p
↪  '{"spec":{"template":{"spec":{"affinity":{"nodeAffinity":{"requiredDuringSchedulingIgnoredDuringExecutio
↪  -n rising
```

- Check the status of the Deployment:

```
kubectl describe deployment test-deployment -n rising
```

- Inspect the Pods:

```
kubectl get pods -l app=test -n rising
```

- Tear everything down:

```
kubectl delete deployment test-deployment -n rising
kubectl delete resourcequota test-quota -n rising
```

3.5 Deployment log D5 - Manage two Deployments contemporaneously

- Create two Deployments together:

```
kubectl apply -f - <<EOF
    apiVersion: apps/v1
    kind: Deployment
    metadata:
      name: interesting-deployment-1
      namespace: rising
      labels:
        app: interesting-app
    spec:
      replicas: 4
      selector:
        matchLabels:
          app: interesting-app
      template:
        metadata:
          labels:
            app: interesting-app
        spec:
          containers:
          - name: interesting-container-1
            image: nginx:1.19.6
            ports:
            - containerPort: 80
          - name: sidecar-container-1
            image: busybox:1.31.1
            command: ['sh', '-c', 'echo sidecar-container-1 is running && sleep 3600']
    ---
    apiVersion: apps/v1
    kind: Deployment
    metadata:
      name: interesting-deployment-2
      namespace: rising
      labels:
        app: interesting-app
    spec:
      replicas: 2
      selector:
        matchLabels:
          app: interesting-app
      template:
        metadata:
          labels:
            app: interesting-app
        spec:
          containers:
          - name: interesting-container-2
            image: nginx:latest
          initContainers:
          - name: init-container-1
            image: busybox:1.31.1
            command: ['sh', '-c', 'echo init-container-1 is running && sleep 5']
    EOF
```

- Wait for the Deployments to be ready:

```
kubectl wait --for=condition=Available deployment/interesting-deployment-1 -n rising
kubectl wait --for=condition=Available deployment/interesting-deployment-2 -n rising
```

- Check the rollout status of the Deployments:

```
kubectl rollout status deployment interesting-deployment-1 -n rising
kubectl rollout status deployment interesting-deployment-2 -n rising
```

- Inspect the Deployments:

```
kubectl get deployments -l app=interesting-app -n rising
```

3.6 ReplicaSet log `RS1` - Use bare ReplicaSets to manage Pods

- Create a ReplicaSet:

```
cat <<EOF | kubectl apply -f -
    apiVersion: apps/v1
    kind: ReplicaSet
    metadata:
      name: frontend
      namespace: rising
      labels:
        app: guestbook
        tier: frontend
    spec:
      replicas: 3
      selector:
        matchLabels:
          tier: frontend
      template:
        metadata:
          labels:
            tier: frontend
        spec:
          containers:
          - name: outward
            image: httpd:latest
    EOF
```

- List the ReplicaSets:

```
kubectl get replicasets -n rising
```

- Inspect the ReplicaSet:

```
kubectl describe replicasets frontend -n rising
```

- Scale down the ReplicaSet:

```
kubectl scale replicasets frontend --replicas=2 -n rising
```

- Inspect the Pods:

```
kubectl get pods -l tier=frontend -n rising
```

- Tear everything down:

```
kubectl delete replicasets frontend -n rising
```

3.7 DaemonSet log `DS1` - Manage a DaemonSet

- Create a DaemonSet:

```
cat <<EOF | kubectl apply -f -
    apiVersion: apps/v1
    kind: DaemonSet
    metadata:
      name: fluentd-elasticsearch
      namespace: kube-system
      labels:
        k8s-app: fluentd-logging
    spec:
      selector:
        matchLabels:
          name: fluentd-elasticsearch
      template:
        metadata:
          labels:
            name: fluentd-elasticsearch
        spec:
          tolerations:
          - key: node-role.kubernetes.io/control-plane
            operator: Exists
            effect: NoSchedule
          containers:
          - name: fluentd-elasticsearch
            image: quay.io/fluentd_elasticsearch/fluentd:v2.5.2
            resources:
              limits:
                memory: 200Mi
              requests:
                cpu: 100m
                memory: 200Mi
            volumeMounts:
            - name: varlog
              mountPath: /var/log
          terminationGracePeriodSeconds: 30
          volumes:
          - name: varlog
            hostPath:
              path: /var/log
    EOF
```

- Inspect the DaemonSet:

```
kubectl get daemonsets fluentd-elasticsearch -n kube-system
```

- Inspect the Pods:

```
kubectl get pods -l name=fluentd-elasticsearch -n kube-system
```

- Cordon `kubeadm-worker1`:

```
kubectl cordon kubeadm-worker1
```

- Check the Pods again:

```
kubectl get pods -l name=fluentd-elasticsearch -n kube-system
```

- Uncordon and add a label to `kubeadm-worker1`:

```
kubectl uncordon kubeadm-worker1
kubectl taint nodes kubeadm-worker1 test-label=no-schedule:NoExecute
```

- Edit the DaemonSet to tolerate the taint, this will drop out the master node:

```
kubectl patch daemonset fluentd-elasticsearch -p
↪  '{"spec":{"template":{"spec":{"tolerations":[{"key":"test-label","operator":"Equal","effect":"NoExecute"
↪  -n kube-system
```

- Check the Pods again:

```
kubectl get pods -l name=fluentd-elasticsearch -n kube-system
```

- Clean everything up:

```
kubectl delete daemonset fluentd-elasticsearch -n kube-system
kubectl taint nodes kubeadm-worker1 test-label-
```

3.8 Job log J1 - Manage a Job

- Create a Job:

```
cat <<EOF | kubectl apply -f -
    apiVersion: batch/v1
    kind: Job
    metadata:
      name: pi
      namespace: rising
    spec:
      template:
        spec:
          containers:
          - name: pi
            image: perl:5.34.0
            command: ["perl",  "-Mbignum=bpi", "-wle", "print bpi(2000)"]
          restartPolicy: Never
      backoffLimit: 4
    EOF
```

- List available Jobs:

```
kubectl get jobs -n rising
```

- Inspect the Job:

```
kubectl describe jobs pi -n rising
```

- Check the Pods:

```
kubectl get pods -l job-name=pi -n rising
```

- Assess the logs generated by the Job:

```
kubectl wait --for=condition=complete job/pi -n rising
kubectl logs jobs/pi -n rising | head -c 20
```

- Update the spec of the Job to trigger auto deletion:

```
kubectl patch job pi -p '{"spec":{"ttlSecondsAfterFinished":60}}' -n rising
```

- Wait for the Job to be deleted:

```
kubectl wait --for=delete job/pi -n rising --timeout=80s
```

3.9 CronJob log `CJ1` - Manage a CronJob

- Create a CronJob:

```
cat <<EOF | kubectl apply -f -
    apiVersion: batch/v1
    kind: CronJob
    metadata:
      name: hello
      labels:
        cj: hello
      namespace: rising
    spec:
      schedule: "* * * * *"
      jobTemplate:
        metadata:
          labels:
            cj: hello
        spec:
          template:
            spec:
              containers:
              - name: hello
                image: busybox:1.28
                imagePullPolicy: IfNotPresent
                command:
                - /bin/sh
                - -c
                - date; echo Hello from the Kubernetes cluster
              restartPolicy: OnFailure
    EOF
```

- List available CronJobs:

```
kubectl get cronjobs -n rising
```

- Inspect the CronJob:

```
kubectl describe cronjobs hello -n rising
```

- Wait two-three minutes and check the Jobs created by the CronJob:

```
sleep 120
kubectl get jobs -l cj=hello -n rising
```

- Clean everything up:

```
kubectl delete cronjob hello -n rising
```

4 Log setups level 1b

4.1 **DONE** Pod log `P3` - Use kubectl debug and Ephemerals Containers

- Run a dumb Pod that does not have a shell:

```
kubectl run -it -n rising --rm --restart=Never --image=registry.k8s.io/pause:latest dumb-pod
↪  -- sh
```

- Try to launch a `kubectl exec`, it will fail:

```
kubectl exec -it dumb-pod -- sh
```

- Use `kubectl debug` to attach an Ephemeral Container:

```
kubectl debug -it ephemeral-dumb-pod --image=busybox:1.31.1 --target=dumb-pod
```

- Describe the newly created Ephemeral Container:

```
kubectl describe pod ephemeral-dumb-pod -n rising
kubectl describe pod dumb-pod -n rising
```

- Destroy the ephemeral Pod and the dumb Pod:

```
kubectl delete pod ephemeral-dumb-pod -n rising
kubectl delete pod dumb-pod -n rising
```

4.2 DONE LimitRange log `LR1` - Create a LimitRange to restrict the resources of a Pod

- Define a LimitRange for the namespace:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: LimitRange
    metadata:
      name: limit-range
      namespace: rising
    spec:
      limits:
      - default        :
          cpu: 300m
          memory: 200Mi
        defaultRequest:
          cpu: 200m
          memory: 100Mi
        max:
          cpu: 500m
          memory: 500Mi
        min:
          cpu: 100m
          memory: 50Mi
    EOF
```

- Inspect the newly created LimitRange:

```
kubectl describe limitrange limit-range -n rising
```

- Create a Pod that does not fit the LimitRange:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Pod
    metadata:
      name: bad-pod
      namespace: rising
    spec:
      containers:
      - name: bad-container
        image: busybox:1.31.1
        command: ["sh", "-c", "sleep 3600"]
        resources:
          requests:
            cpu: 600m
            memory: 300Mi
    EOF
```

- Inspect the offending Pod:

```
kubectl describe pod bad-pod -n rising
```

- Delete it:

```
kubectl delete pod bad-pod -n rising
```

- Create a lawful Pod:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Pod
    metadata:
      name: good-pod
      namespace: rising
    spec:
      containers:
      - name: good-container
        image: busybox:1.31.1
        command: ["sh", "-c", "sleep 3600"]
        resources:
          requests:
            cpu: 200m
            memory: 100Mi
    EOF
```

- Inspect the good Pod:

```
kubectl describe pod good-pod -n rising
```

- Delete it:

```
kubectl delete pod good-pod -n rising
```

- Clean everything up:

```
kubectl delete limitrange limit-range -n rising
```

5 Log setups level 3

5.1 DONE Service log SV1 - Expose the three types of Services from a Pod

- Create a Pod:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Pod
    metadata:
      name: service-pod
      namespace: rising
      labels:
        app: service-app
    spec:
      containers:
      - name: service-container
        image: nginx:1.19.6
        ports:
          - containerPort: 80
            name: http
    EOF
```

- Expose a ClusterIP service (by default if you don't specify its type):

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Service
    metadata:
      name: nginx-service
      namespace: rising
    spec:
      selector:
        app: service-app
      ports:
      - name: http-external
        protocol: TCP
        port: 80
        targetPort: http
    EOF
```

- Inspect the newly create Service:

```
kubectl describe service nginx-service -n rising
```

- List the Services:

```
kubectl get services -n rising
```

- Update the Service, and replace it with a NodePort:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Service
    metadata:
      name: nginx-service
      namespace: rising
    spec:
      type: NodePort
      selector:
        app: service-app
      ports:
      - name: http-external
        protocol: TCP
        port: 80
        targetPort: http
        nodePort: 30080
    EOF
```

- Inspect the updated Service:

```
kubectl describe service nginx-service -n rising
```

- List Services again:

```
kubectl get services -n rising
```

LoadBalancer services are not supported in my environment, so I can't test them.

- Add an ExternalName service:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Service
    metadata:
      name: external-service
      namespace: rising
    spec:
      type: ExternalName
      externalName: risinglab.fbk.eu
    EOF
```

- Clean all the resources:

```
kubectl delete pod service-pod -n rising
kubectl delete service nginx-service -n rising
kubectl delete service external-service -n rising
```

5.2 DONE Service log SV2 - Expose Deployments through Services

- Create a Deployment with an explicit containerPort:

```
cat <<EOF | kubectl apply -f -
    apiVersion: apps/v1
    kind: Deployment
    metadata:
      name: exposed-deployment
      namespace: rising
    spec:
      selector:
        matchLabels:
          run: nginx-deployment
      replicas: 2
      template:
        metadata:
          labels:
            run: nginx-deployment
        spec:
          containers:
          - name: nginx-runner
            image: nginx
            ports:
            - containerPort: 80
    EOF
```

- Check the newly created Deployment:

```
kubectl get deployments -n rising -o wide
```

- Expose the Deployment, it will default to a ClusterIP using the only containerPort available:

```
kubectl expose deployment exposed-deployment
```

- List the Services:

```
kubectl get services -n rising
```

- Describe the Service:

```
kubectl describe service exposed-deployment -n rising
```

- List the EndpointSlices:

```
kubectl get endpointslices -n rising
```

- Scale the Deployment:

```
kubectl scale deployment exposed-deployment --replicas=3 -n rising
```

- Inspect the changes in the Service:

```
kubectl describe service exposed-deployment -n rising
```

- Clean everything up:

```
kubectl delete deployment exposed-deployment -n rising
kubectl delete service exposed-deployment -n rising
```

5.3 DONE Service log SV3 - Use multi-port Services

- Create a Pod that exposes two ports:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Pod
    metadata:
      name: multi-port-pod
      namespace: rising
      labels:
        app: multi-port-app
    spec:
      containers:
      - name: multi-port-container
        image: nginx:1.19.6
        ports:
        - containerPort: 80
          name: http
        - containerPort: 443
          name: https
    EOF
```

- Expose the Pod through a Service:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Service
    metadata:
      name: multi-port-service
      namespace: rising
    spec:
      selector:
        app: multi-port-app
      ports:
      - name: http
        protocol: TCP
        port: 80
        targetPort: http
      - name: https
        protocol: TCP
        port: 443
        targetPort: https
    EOF
```

- Inspect the Service:

```
kubectl describe service multi-port-service -n rising
```

- Clean everything up:

```
kubectl delete pod multi-port-pod -n rising
kubectl delete service multi-port-service -n rising
```

5.4 DONE EndpointSlice log ES1 - Use EndpointSlices to manually group selectorless Services

- Create two Pods and a Service that has no selector:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Pod
    metadata:
      name: endpoint-pod-1
      namespace: rising
    spec:
      containers:
      - name: endpoint-container
        image: nginx:1.19.6
        ports:
        - containerPort: 80
          name: http
    ---
    apiVersion: v1
    kind: Pod
    metadata:
      name: endpoint-pod-2
      namespace: rising
    spec:
      containers:
      - name: endpoint-container
        image: nginx:1.27.0
        ports:
        - containerPort: 80
          name: http
    ---
    apiVersion: v1
    kind: Service
    metadata:
      name: endpoint-service
      namespace: rising
    spec:
      ports:
        - name: http
          port: 80
          targetPort: 4000
    EOF
```

- Inspect EndpointSlices:

```
kubectl get endpointslices -n rising
```

- This Service will not create any EndpointSlice. We need to create one manually:

```
cat <<EOF | kubectl apply -f -
    apiVersion: discovery.k8s.io/v1
    kind: EndpointSlice
    metadata:
      name: endpoint-service-1
      namespace: rising
      labels:
        kubernetes.io/service-name: endpoint-service
    addressType: IPv4
    ports:
      - name: http
        appProtocol: http
        protocol: TCP
        port: 4000
    endpoints:
        - addresses:
            - $(kubectl get pod endpoint-pod-1 -n rising -o jsonpath='{.status.podIP}')
        - addresses:
            - $(kubectl get pod endpoint-pod-2 -n rising -o jsonpath='{.status.podIP}')
    EOF
```

- Inspect the EndpointSlice:

```
kubectl describe endpointslice endpoint-service-1 -n rising
```

- Tear everything down:

```
kubectl delete pod endpoint-pod-1 -n rising
kubectl delete pod endpoint-pod-2 -n rising
kubectl delete service endpoint-service -n rising
kubectl delete endpointslice endpoint-service-1 -n rising
```

5.5 SCHE Ingress log IG1 - Import an external IngressController, then create an Ingress and expose a Service

- Import the NGINX Ingress Controller:

```
kubectl apply -f
↪  https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/kind/deploy.yaml
```

- Deploy a Pod and a Service:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Pod
    metadata:
      name: ingress-pod
      namespace: rising
    spec:
      containers:
      - name: ingress-container
        image: nginx:1.19.6
        ports:
        - containerPort: 80
          name: http
    ---
    apiVersion: v1
    kind: Service
    metadata:
      name: ingress-service
      namespace: rising
    spec:
      selector:
        app: ingress-app
      ports:
      - name: http
        protocol: TCP
        port: 80
        targetPort: http
    EOF
```

- Create the Ingress:

```
cat <<EOF | kubectl apply -f -
    apiVersion: networking.k8s.io/v1
    kind: Ingress
    metadata:
      name: ingress
      namespace: rising
      annotations:
        nginx.ingress.kubernetes.io/rewrite-target: /
    spec:
      rules:
      - host: ingress.local
        http:
          paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web
                port:
                  number: 80
    EOF
```

- Inspect the Ingress:

```
kubectl describe ingress ingress -n rising
```

- List the IngressClasses:

```
kubectl get ingressclasses -n rising
```

- Clean what we created:

```
kubectl delete pod ingress-pod -n rising
kubectl delete service ingress-service -n rising
kubectl delete ingress ingress -n rising
```

- Remove the NGINX Ingress Controller:

```
kubectl delete -f
↪  https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/kind/deploy.yaml
```

5.6 SCHE Pod Networking log PN1 - Use Port Forwarding to access a Pod

- Create a Deployment:

```
cat <<EOF | kubectl apply -f -
    apiVersion: apps/v1
    kind: Deployment
    metadata:
      name: mongo
      labels:
        app.kubernetes.io/name: mongo
        app.kubernetes.io/component: backend
    spec:
      selector:
        matchLabels:
          app.kubernetes.io/name: mongo
          app.kubernetes.io/component: backend
      replicas: 1
      template:
        metadata:
          labels:
            app.kubernetes.io/name: mongo
            app.kubernetes.io/component: backend
        spec:
          containers:
          - name: mongo
            image: mongo:4.2
            args:
              - --bind_ip
              - 0.0.0.0
            resources:
              requests:
                cpu: 100m
                memory: 100Mi
            ports:
            - containerPort: 27017
    EOF
```

- Inspect Deployments, ReplicaSets, Pods:

```
kubectl get deployments -n rising
kubectl get replicasets -n rising
kubectl get pods -n rising
```

- Create a Service to expose Mongo:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Service
    metadata:
      name: mongo
      labels:
        app.kubernetes.io/name: mongo
        app.kubernetes.io/component: backend
    spec:
      ports:
      - port: 27017
        targetPort: 27017
      selector:
        app.kubernetes.io/name: mongo
        app.kubernetes.io/component: backend
    EOF
```

- Port-forward the Pod and send the process to background:

  ```
  kubectl port-forward deployment/mongo 28015:27017 &
  pid=$!
  ```

- Clean everything up:

```
kill $pid
kubectl delete deployment mongo -n rising
kubectl delete service mongo -n rising
```

5.7 SCHE Proxy Log PX1 - Manually construct a Proxy API call for Nodes, Pods, and Services

- Use `kubectl cluster-info` to obtain endpoint information:

```
cplane=$(kubectl cluster-info | grep 'control plane' | grep -oP 'https://[^/]+' --color=never)
proxy_endpoint=${cplane}
echo $proxy_endpoint
```

- Try the node proxy (the response will be empty, but the call will be successful):

```
first_node=$(kubectl get nodes -o jsonpath='{.items[0].metadata.name}')
./scripts/kube-api.sh $proxy_endpoint api/v1/nodes/${first_node}:10250/proxy
```

- Deploy a Pod and a Service:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Pod
    metadata:
      name: proxy-pod
      namespace: rising
    spec:
      containers:
      - name: nginx
        image: nginx
        ports:
        - containerPort: 80
    ---
    apiVersion: v1
    kind: Service
    metadata:
      name: proxy
      namespace: rising
    spec:
      selector:
        app: proxy
      ports:
      - protocol: TCP
        port: 80
        targetPort: 80
    EOF
```

- Try the pod proxy:

```
first_pod=$(kubectl get pods -n rising -o jsonpath='{.items[0].metadata.name}')
echo ./scripts/kube-api.sh $proxy_endpoint api/v1/namespaces/rising/pods/${first_pod}:80/proxy
```

- Clean everything up:

```
kubectl delete pod proxy-pod -n rising
kubectl delete service proxy -n rising
```

5.8 SCHE NetworkPolicy log NP1 - Create a NetworkPolicy to restrict traffic to a Pod

- Run some services:

```
kubectl run web1 --image=nginx --port=80 --labels="app=web" --expose --namespace=default
kubectl run web2 --image=nginx --port=80 --labels="app=web" --expose --namespace=rising
kubectl run bb --image=busybox --labels="app=bb" --namespace=rising -- sleep 3600
```

A pod is isolated for ingress if there is any NetworkPolicy that both selects the pod and has "Ingress" in its policyTypes. A pod is isolated for egress if there is any NetworkPolicy that both selects the pod and has "Egress" in its policyTypes.

- Create a NetworkPolicy:

```
cat <<EOF | kubectl apply -f -
    apiVersion: networking.k8s.io/v1
    kind: NetworkPolicy
    metadata:
      name: bb-to-web
      namespace: rising
    spec:
      podSelector:
        matchLabels:
          app: web
      policyTypes:
      - Ingress
      ingress:
      - from:
        - podSelector:
            matchLabels:
              app: bb
    ---
    apiVersion: networking.k8s.io/v1
    kind: NetworkPolicy
    metadata:
      name: web-to-web-block
      namespace: rising
    spec:
      podSelector: {}
      policyTypes:
      - Ingress
      - Egress
    ---
    apiVersion: networking.k8s.io/v1
    kind: NetworkPolicy
    metadata:
      name: web-to-web-block
      namespace: default
    spec:
      podSelector: {}
      policyTypes:
      - Ingress
      - Egress
    EOF
```

- Assess the NetworkPolicy:

```
kubectl get networkpolicies -A -o wide
kubectl describe networkpolicy bb-to-web -n rising
```

- See if the busybox Pod can reach the web1 Pod:

```
kubectl exec -n rising bb -- wget -qO- http://web1.default | grep Welcome
```

- See if the web1 Pod can connect to the web2 Pod:

```
kubectl exec -n default web1 -- curl -s web2.rising
```

- Clean everything up:

```
kubectl delete networkpolicy bb-to-web web-to-web-block -n rising
kubectl delete networkpolicy web-to-web-block -n default
kubectl delete pod web1 -n default
kubectl delete pod web2 bb -n rising
kubectl delete service web1 -n default
kubectl delete service web2 -n rising
```

6 Log setups level 4

6.1 **WAIT** Stateful Set log `SS1` - Manage a StatefulSet

- Create a dumb StorageClass for the StatefulSet:

```
cat <<EOF | kubectl apply -f -
    apiVersion: storage.k8s.io/v1
    kind: StorageClass
    metadata:
      name: local-storage
      annotations:
        storageclass.kubernetes.io/is-default-class: "true"
    provisioner: kubernetes.io/no-provisioner
    volumeBindingMode: WaitForFirstConsumer
    EOF
```

- Create some PersistentVolumes:

```
  cat <<EOF | kubectl apply -f -
      apiVersion: v1
      kind: PersistentVolume
      metadata:
        name: pv-0
      spec:
        capacity:
          storage: 1Gi
        volumeMode: Filesystem
        accessModes:
          - ReadWriteOnce
        persistentVolumeReclaimPolicy: Retain
        storageClassName: local-storage
        local:
    EOF
```

```
The PersistentVolume "pv-0" is invalid: spec: Required value: must specify a volume type
```

- Create a StatefulSet with an associated Service:

```
cat <<EOF | kubectl apply -f -
    apiVersion: v1
    kind: Service
    metadata:
      name: nginx
      namespace: rising
      labels:
        app: nginx
    spec:
      ports:
      - port: 80
        name: web
      clusterIP: None
      selector:
        app: nginx
    ---
    apiVersion: apps/v1
    kind: StatefulSet
    metadata:
      namespace: rising
      name: web
    spec:
      selector:
        matchLabels:
          app: nginx
      serviceName: "nginx"
      replicas: 3
      minReadySeconds: 10 # by default is 0
      template:
        metadata:
          labels:
            app: nginx
        spec:
          terminationGracePeriodSeconds: 10
          containers:
          - name: nginx
            image: registry.k8s.io/nginx-slim:0.24
            ports:
            - containerPort: 80
              name: web
            volumeMounts:
            - name: www
              mountPath: /usr/share/nginx/html
      volumeClaimTemplates:
      - metadata:
          name: www
        spec:
          accessModes: [ "ReadWriteOnce" ]
          resources:
            requests:
              storage: 200Mi
    EOF
```

- Inspect the StatefulSet:

```
kubectl get statefulsets web -n rising
```

- Inspect the Pods:

```
kubectl get pods -l app=nginx -n rising
```

- Scale up the StatefulSet:

```
kubectl scale statefulsets web --replicas=4 -n rising
```

- Inspect the Pods again:

```
kubectl get pods -l app=nginx -n rising
```

- Scale down to 2:

```
kubectl scale statefulsets web --replicas=2 -n rising
```

7 Real-life scenarios (level infinity)

7.1 **WAIT** Real-life scenario `RL1` - Deploy the NGINX Ingress Controller and expose a Deployment through it